Recipe planner- lab report

1. System Overview

The application is a fullstack web app for creating and managing recipes with automatically calculated nutritional values. Users can log in, create recipes, and assign multiple ingredients with specific amounts. The system calculates total calories, protein, carbohydrates, and fat based on ingredient data. Each recipe is linked to a user, making it possible to track who created what. The app is designed for individuals who want to plan meals and understand their nutritional intake. The frontend is built with React, the backend with Express, and data is stored in MongoDB Atlas.

2. Database Design

The application uses three main collections: Users, Ingredients, and Recipes.

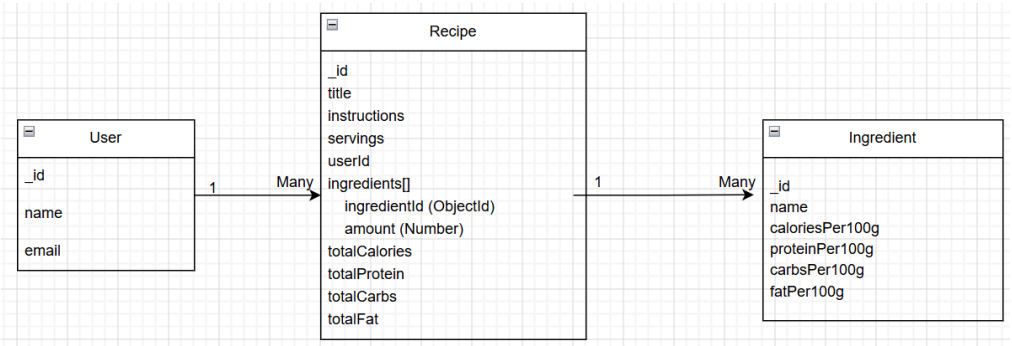

Figure 1: ERD diagram showing the relationship between User, Recipe, and Ingredient collections.

Figure 1 shows the ERD for the application, including the relationships between Users, Recipes, and Ingredients.

Relationships (ERD)

- A Recipe belongs to one User
  → recipe.userId → user._id
- A Recipe contains multiple Ingredients
  → recipe.ingredients[].ingredientId → ingredient._id

This structure allows recipes to include multiple ingredients and enables population of related data when fetching recipes.

3. API Endpoints

1. Create Recipe

Route: POST /api/recipes
Method: POST
Request Body:

```
{
  "title": "Chicken Rice Bowl",
  "instructions": "Cook rice and chicken",
```

```
  "servings": 2,
  "userId": "user_id",
  "ingredients": [
    { "ingredientId": "ingredient_id", "amount": 200 }
  ]
}
```

Response:

```
{
  "_id": "recipe_id",
  "title": "Chicken Rice Bowl",
  "totalCalories": 600
}
```

2. Get Recipes

Route: GET /api/recipes
Method: GET
Response:

```
[
  {
    "title": "Chicken Rice Bowl",
    "servings": 2,
    "totalCalories": 600,
    "userId": { "name": "Oskar Nilsson" },
    "ingredients": [
      { "ingredientId": { "name": "Chicken" }, "amount": 200 }
    ]
  }
]
```

This endpoint uses populate() to include related user and ingredient data.

4. Reflection

One challenge I faced was related to the login functionality. When testing the login flow, the application sometimes rendered a blank page instead of the login form. This issue was difficult to debug because there were no clear error messages in the UI. The root cause was that invalid data from the backend (an error response) was being stored as a user in localStorage. The application then assumed a valid user was logged in, but the object did not contain expected properties such as name, which caused the React components to break. Additionally, there was a structural bug in the App component where rendering logic was mistakenly placed inside a helper function instead of the main return statement, preventing React from rendering anything. The issue was resolved by validating the backend response before setting the user, preventing invalid data from being stored in localStorage, and refactoring the App component so that all rendering logic was placed correctly in the return statement. I also ensured that conditional rendering (if (!user)) properly returned the Login component. From this, I learned the importance of validating data from the backend and structuring React components correctly. I also learned how small mistakes in state management and rendering logic can completely break the UI.

5. Feature Iteration

One feature that evolved during development was the Create Recipe form. Initially, the form was implemented using controlled inputs and only logged the entered data to the console. This helped verify that state management and input handling were working correctly. Later, the feature was improved by connecting it to the backend API. The form was updated to send a POST request to /api/recipes, allowing recipes to be saved in the database. Additional improvements included support for multiple ingredients, dynamic input fields (add/remove ingredient rows), and automatic calculation of nutritional values on the backend. This iteration demonstrates a clear progression from a basic frontend-only implementation to a fully integrated fullstack feature.
Example commits:

5cd00f2- feat: add basic create recipe form with controlled inputs
bc37caf- feat: improve create recipe form to send data to backend